PROJECT SOURCE CODE

```
#Pothole Object Detection
import torchvision.transforms.functional as tf
from utils import get_pothole_model,detection_img

st.title("Pothole Object Detection")
classes=["Background","Pothole"]
dirname=os.path.dirname(os.path.abspath(__file__))
root_dir=os.path.join(dirname,os.pardir)
pothole_img_dir=os.path.join(root_dir,"images","pothole_img")
model=get_pothole_model()
model.eval()
file=st.file_uploader("Input Image File",type = ['jpg','png','jpeg'])
c1,c2=st.columns(2)
with c1:
    conf_threshold=float(st.slider("Confidence Threshold", min_value=0.0,max_value=1.0,value=0.2,
        step=0.02))
with c2:
    iou_threshold=float(st.slider("IOU Threshold", min_value=0.0,max_value=1.0,value=0.7,step=0.02))
col1,col2=st.columns(2)
button=st.button("Detect")
if button:
    if file is not None:
        title="Uploaded Image"
        img=Image.open(file)
        img=np.array(img)
        img=cv2.resize(img,(480,480))
    else:
        title="Default Image"
        idx=np.random.choice(range(4),1)[0]
        default_img_path=os.path.join(pothole_img_dir,f"{idx}.png")
        img=Image.open(default_img_path)
        img=np.array(img)
        img=cv2.resize(img,(480,480))
    with col1:
        st.write(title)
```

```
        st.image(img)
    with col2:
        st.write("Pothole Object Detection")
        detect_img=detection_img(model,img,classes,conf_threshold,iou_threshold)
        st.image(detect_img)

#Traffic Density Classification
from PIL import Image
import cv2
from utils import classify_img,get_density_model

st.title("Traffic Density Classification")

classes=['Empty', 'High', 'Low', 'Medium', 'Traffic Jam']
model=get_density_model()
model.eval()
dirname=os.path.dirname(os.path.abspath(__file__))
root_dir=os.path.join(dirname,os.pardir)
density_img_dir=os.path.join(root_dir,"images","density img")

file=st.file_uploader("Input Image File",type = ['jpg','png','jpeg'])
button=st.button("Detect")
if button:
    if file is not None:
        title="Uploaded Image"
        img=Image.open(file)
        img=np.array(img)
        img=cv2.resize(img,(480,480))
        label,probability=classify_img(model,img)
    else:
        title="Default Image"
        idx=np.random.choice(range(5),1)[0]
        default_img_path=os.path.join(density_img_dir,f"{idx}.jpg")
        img=Image.open(default_img_path)
        img=np.array(img)
        img=cv2.resize(img,(480,480))
        label,probability=classify_img(model,img)
    st.success(f"Predicted Class is {classes[label]} with probability {probability:.4f}")
    st.image(img)

#Vehicle Category Detection
import streamlit as st
from PIL import Image
import cv2
import numpy as np
import os
import torch
import torchvision
from utils import get_category_model,detection_img

```

```
st.title("Vehicle Category Detection")

classes=["background","Auto","Bus","Car","LCV","Motorcycle","Truck","Tractor","Multi-Axle"]
model=get_category_model()
model.eval()
dirname=os.path.dirname(os.path.abspath(__file__))
root_dir=os.path.join(dirname,os.pardir)
category_img_dir=os.path.join(root_dir,"images","category_img")

file=st.file_uploader("Input Image File",type = ['jpg','png','jpeg'])
c1,c2=st.columns(2)
with c1:
    conf_threshold=float(st.slider("Confidence Threshold", min_value=0.0,max_value=1.0,value=0.2,
        step=0.02))
with c2:
    iou_threshold=float(st.slider("IOU Threshold", min_value=0.0,max_value=1.0,value=0.6,step=0.02))
col1,col2=st.columns(2)
button=st.button("Detect")
if button:
    if file is not None:
        title="Uploaded Image"
        img=Image.open(file)
        img=np.array(img)
        img=cv2.resize(img,(480,480))
    else:
        title="Default Image"
        idx=np.random.choice(range(7),1)[0]
        default_img_path=os.path.join(category_img_dir,f"{idx}.jpg")
        img=Image.open(default_img_path)
        img=np.array(img)
        img=cv2.resize(img,(480,480))
    with col1:
        st.write(title)
        st.image(img)
    with col2:
        st.write("Vehicle Category Detection")
        detect_img=detection_img(model,img,classes,conf_threshold,iou_threshold)
        st.image(detect_img)
```